GRC Weekly Report Pipeline - POC Implementation Plan

Context

Alphanumeric produces a weekly GRC report for GSK's chatbot (powered by Inbenta). Today this is a manual, ~2-3 hour process involving: exporting 3 datasets from Inbenta's dashboard, loading them into a macro-enabled Excel template, performing ~20 manual steps of data cleaning/joining/pivoting across 5 sheets, then manually classifying each session. The goal is to build a Python pipeline that automates the data extraction, transformation, and report generation — reducing analyst effort to only the manual classification step.

Input: Inbenta Reporting API (4 event endpoints + aggregates)

Output: .xlsm file matching the exact format of the reference CSV at /Users/murlialva/Personal/DAW/04 GRC Weekly report 011826_012426.csv

Trigger: On-demand via CLI command (analyst runs when needed). Optional cron config available for future scheduled automation.

Architecture Overview

```
config.yaml (credentials, excluded sources, date range)
     |

┌──────────────────────┐
|  1. API CLIENT       |  Auth + HMAC signing + pagination + rate limiting
|  (src/api/)          |  Endpoints: user_questions, sessions, clicks, ratings,
|                      |  session_details, event_sources, session_data_keys
└──────────┬───────────┘

┌──────────────────────┐
|  2. INGESTION        |  Fetch all data for a Sun-Sat week, store in SQLite
|  (src/ingestion/)    |  Idempotent: delete+re-insert per week_key
└──────────┬───────────┘

┌──────────────────────┐
|  3. TRANSFORM        |  Filter sources → extract SEARCH events → join
|  (src/transform/)    |  questions to sessions → resolve content titles →
|                      |  build hierarchical Session/Question/Answer/Rating rows
└──────────┬───────────┘

┌──────────────────────┐
|  4. EXCEL OUTPUT     |  Write .xlsm with openpyxl: data + styling +
|  (src/output/)       |  dropdown validations for manual columns
└──────────────────────┘
```

Project Structure

```
grc-pipeline/
├── config/
|   └── config.yaml              # API creds (env var refs), filters, output settings
├── src/
|   ├── __init__.py
|   ├── cli.py                   # Click-based CLI: ingest, transform, generate, full-run
```

```
|   ├── api/
|   |   ├── __init__.py
|   |   ├── auth.py              # POST /v1/auth → token caching + refresh
|   |   ├── signing.py           # HMAC-SHA256 request signing (v1 protocol)
|   |   └── client.py            # InbentaReportingClient: paginated fetch, rate limit
|   ├── db/
|   |   ├── __init__.py
|   |   ├── engine.py            # SQLite connection, migration runner
|   |   ├── schema.py            # All CREATE TABLE statements
|   |   └── repository.py        # Bulk insert/upsert per table
|   ├── ingestion/
|   |   ├── __init__.py
|   |   ├── pipeline.py          # Orchestrates endpoint→table ingestion
|   |   └── extractors.py        # Per-endpoint JSON→DB row normalization
|   ├── transform/
|   |   ├── __init__.py
|   |   ├── session_builder.py   # Core: builds hierarchical report rows
|   |   ├── filters.py           # Source exclusion (configurable, case-insensitive)
|   |   └── content_resolver.py  # id_content → intent title lookup
|   ├── output/
|   |   ├── __init__.py
|   |   └── excel_writer.py      # openpyxl .xlsm generation with styling + validation
|   └── utils/
|       ├── __init__.py
|       ├── logging_config.py
|       └── date_utils.py        # Week range computation, timezone handling
├── tests/
|   ├── fixtures/                # Sample API response JSONs
|   ├── test_api_client.py
|   ├── test_signing.py
|   ├── test_transform.py
|   ├── test_excel_writer.py
|   └── test_integration.py     # Validate against reference CSV
├── data/                        # .gitignored runtime artifacts
|   ├── grc_pipeline.db
|   └── reports/
├── requirements.txt
└── pyproject.toml
```

Implementation Steps

Step 1: Project Skeleton & Config

Files: pyproject.toml, requirements.txt, config/config.yaml, src/cli.py, src/utils/

- Initialize Python project with dependencies: requests, click, openpyxl, pyyaml, tenacity, python-dotenv
- Config loader that resolves ${ENV_VAR} references for secrets
- Click CLI skeleton with commands: ingest, transform, generate-report, full-run
- Structured logging (console + rotating file)

Step 2: Authentication & HMAC Signing

Files: src/api/auth.py, src/api/signing.py

Auth flow (per Inbenta docs):

- POST https://api.inbenta.io/v1/auth with header x-inbenta-key and body {"secret": "..."}

- Response: {"accessToken": "...", "expiration": 1200, ...}
- Cache token, refresh 2 min before expiry
- Never refresh after every call (auth rate limit: 10 req/min on POC tier)

HMAC-SHA256 signing (per Inbenta docs):

- Base string = METHOD&percent_encode(path_from_/v1)&percent_encode(sorted_query_string)&percent_encode(body)&timestamp&v1
- Empty elements excluded
- Headers: x-inbenta-signature, x-inbenta-signature-version: v1, x-inbenta-timestamp

Step 3: API Client with Pagination

Files: src/api/client.py

- Base URL: https://api-reporting-us.inbenta.io
- Pagination: offset/length (max 1000), loop until offset >= total_count
- Date windowing: Split 7-day range into ≤3-day windows (API recommends this to avoid 504 timeouts)
- Rate limiting: respect tier limits (800 req/min for product APIs), simple sleep-based throttle
- Retry: tenacity with exponential backoff on 429/500/502/503/504
- source filter: pass allowed sources as JSON array parameter to reduce data transferred

Step 4: Database Schema

Files: src/db/schema.py, src/db/engine.py, src/db/repository.py

Key tables (SQLite):

Table	Source Endpoint	Purpose
event_sources	/aggributes/event_sources	Source name lookup + exclusion flags
session_data_keys	/aggributes/session_data_keys	Key ID→name mapping
raw_user_questions	/events/user_questions	Questions with matching metadata
raw_uq_matchings	(child of above)	Normalized matchings array: id_content, weight, externa
raw_sessions	/events/sessions	Session event timeline (key, value, user_question)
raw_clicks	/events/clicks	Click events (id_content, click_type, log_id)
raw_ratings	/events/ratings	Rating events (rating Yes/No, comment, log_id)
agg_session_details	/aggregates/session_details	One row per session: session_id, source, duration, data
content_lookup	(derived)	id_content → intent title mapping
ingestion_runs	(internal)	Run tracking: status, date range, record counts

All tables carry week_key (e.g., 2026-W04) for idempotent re-ingestion and efficient querying.

Step 5: Ingestion Pipeline

Files: src/ingestion/pipeline.py, src/ingestion/extractors.py

Endpoint calling order:

1. Reference data (parallel): event_sources, session_data_keys

2. Compute source exclusion flags (case-insensitive substring match against configurable list: dev, viiv, GSK_MEDICAL, blenrep, sotrovimab)

3. Aggregate: session_details (filtered to env=production + allowed sources)

4. Raw events (parallel): user_questions, sessions, clicks, ratings

Idempotency: DELETE existing rows for target week_key → INSERT new data, all in one transaction.

Extractor logic: Normalize each API response into flat DB rows. For user_questions, flatten the matchings[] array into raw_uq_matchings child rows.

Step 6: Transformation (Core Logic)

Files: src/transform/session_builder.py, src/transform/filters.py, src/transform/content_resolver.py

This is the most complex step. It replicates the manual Excel process:

6a. Identify sessions: Query agg_session_details for the target week, excluding filtered sources.

6b. Extract SEARCH questions per session:

```
SELECT user_question, date, log_id FROM raw_sessions
WHERE session_id = ? AND key = 'SEARCH' ORDER BY date ASC
```

Group consecutive identical questions (handles the "count > 1" case, e.g., "agent" typed twice).

6c. Resolve answers (matchings) per question:

- Join raw_sessions.log_id → raw_user_questions.event_id (or via shared log reference)
- Get raw_uq_matchings for that event → resolve id_content via content_lookup
- Count clicks per content by joining raw_clicks on log_id + id_content
- Format: '- Intent Name (ID: 123, External: No, Clicks: 0) (multi-line, leading single-quote)
- Zero matchings → output 0

6d. Resolve ratings per question:

- Join raw_ratings on log_id matching the question's log references
- Format: Rating: Yes or Rating: No or empty string

6e. Build hierarchical rows: For each session, output:

```
[Session row]  session_id | total_q_count | "Session" | source | ... | date | time
[Question 1]   text       | count         | "Question"| ...
[Answer 1]     formatted  | count         | "Answer"  | ...
[Rating 1]     text       | count         | "Rating"  | ...
[Question 2]   ...        | ...           | ...
...
```

6f. Content title resolution strategy:

1. Bootstrap from reference CSV (parse existing answer strings to build id→title map)

2. Populate from user_questions matchings if API includes titles

3. Fallback: call Inbenta Knowledge API /v1/contents/{id} and cache result

Step 7: Excel Output Generation

Files: src/output/excel_writer.py

Using openpyxl:

- Headers: Row Labels, Count of User question, Labels, Source, Category, Classification, Quality, Answer available in the KB?, Escalated Sessions, Ratings assessment, Date, Time

- Data validation dropdowns (for analyst manual classification):
  - Category (col E): RC, FA, PQ, LA, N/A
  - Classification (col F): Valid, Invalid
  - Quality (col G): Success, Unsuccessful, Invalid
  - Answer accuracy (col G on Answer rows): Accurate, Inaccurate, No Answer
  - Note: Quality column serves dual purpose — "Quality" on Session rows, "Accuracy" on Answer rows
- Styling: Session rows highlighted blue, frozen header, auto-filter, appropriate column widths
- Save as .xlsm: openpyxl supports this extension. If VBA macros are needed later, use a template approach (load_workbook(template, keep_vba=True))
- Filename: {NN} GRC Weekly report {DDMMYYYY}_{DDMMYYYY}.xlsm

Step 8: Validation Against Reference CSV

Files: tests/test_integration.py

Compare pipeline output against /Users/murlialva/Personal/DAW/04 GRC Weekly report 011826_012426.csv:

- Verify all 350 sessions present with correct session IDs
- Verify question counts per session match
- Verify answer formatting (content IDs, click counts, order)
- Verify source attribution per session
- Verify date/time per session
- Report match percentage and specific mismatches

CLI Usage (On-Demand)

The pipeline is designed for on-demand execution — the analyst runs a CLI command when they need a weekly report. No background process or scheduler is required.

```
# Full weekly run (most common usage – single command does everything)
python -m src.cli full-run --date-from 2026-01-18 --date-to 2026-01-24 --report-num 4

# Individual steps (for debugging/re-runs)
python -m src.cli ingest --date-from 2026-01-18 --date-to 2026-01-24
python -m src.cli transform --week-key 2026-W04
python -m src.cli generate-report --week-key 2026-W04 --report-num 4

# Utilities
python -m src.cli list-sources          # Show all sources + exclusion status
python -m src.cli validate --week-key 2026-W04 --reference path/to/ref.csv
```

Optional: Scheduled Automation (cron)

For future automation, a cron job can be configured to run the pipeline every Monday morning:

```
# crontab -e (run every Monday at 7:00 AM)
0 7 * * 1 cd /path/to/grc-pipeline && python -m src.cli full-run --auto-week --report-num auto
```

The --auto-week flag (optional enhancement) automatically computes the previous Sun-Sat range. The --report-num auto increments the report number based on the last generated report. This is purely optional and can be enabled when ready.

Key Risks & Mitigations

Risk	Mitigation
Session↔question join fails (no reliable log_id link)	Implement 3 join strategies: log_id match, then questio
Content titles not in Reporting API	Bootstrap from reference CSV. Add Knowledge API fallbac
API data archived after 100 days	Document this constraint. For historical data, support
HMAC signing implementation mismatch	Test against the PHP example in Inbenta docs. Use known
Rate limits hit (800 req/min product tier)	Sleep-based throttle, exponential backoff on 429, 3-day

Dependencies

```
requests>=2.31.0       # HTTP client
click>=8.1.0           # CLI framework
openpyxl>=3.1.0        # Excel .xlsm generation
pyyaml>=6.0            # Config parsing
tenacity>=8.2.0        # Retry logic
python-dotenv>=1.0.0   # Environment variable loading
```

SQLite via Python's built-in sqlite3. No ORM needed at this scale.

Build Order (Implementation Sequence)

1. Skeleton: Project structure, config, CLI, logging (~1 session)
2. Auth + Signing: Authentication flow, HMAC signing, test with real API (~1 session)
3. API Client: Paginated fetching, date windowing, rate limiting (~1 session)
4. DB + Ingestion: Schema creation, extractors, first real data pull (~1-2 sessions)
5. Transform: Session builder, answer formatting, rating resolution (~2-3 sessions)
6. Excel Output: .xlsm generation with styling and validation dropdowns (~1 session)
7. Integration Test: Validate against reference CSV, fix mismatches (~1-2 sessions)

Verification Plan

1. Unit tests: Signing, pagination, extractors, answer formatting, date utilities
2. Integration test: Full pipeline run → compare output against reference CSV
3. Manual spot-check: Open generated .xlsm in Excel, verify dropdowns work, verify formatting matches existing reports
4. Data completeness: Compare session counts and question counts between API response totals and DB/report totals